Reviewer Pack — Spectral Gap Determines Tseitin Resolution Length

Entry 815cd97daf8d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 12:51:38 UTC

Spectral Gap Determines Tseitin Resolution Length

Entry ID: `815cd97daf8d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 12:51:38 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): Resolution Proof Length

Statement:

> For a connected graph G with second-largest eigenvalue λ_2 of its adjacency matrix, the resolution proof length of its Tseitin formula is ≥ 2^{Ω(λ_2 n)} for n ≤ 40.

Rationale (proposer's reasoning):

> Spectral gap (λ_2) captures expansion properties critical for resolution hardness. Tseitin formulas on expanders require long proofs, and λ_2 directly quantifies this expansion, offering a concrete link to resolution complexity.

Taxonomy category: `TSEITIN_RES` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e599ff404c1be290`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    p = 0.5

    # Generate Erdős–Rényi graph
    adjacency_matrix = [[random.random() < p for _ in range(n)] for _ in range(n)]
    for i in range(n):
        adjacency_matrix[i][i] = 0

    # Compute Laplacian matrix
    laplacian = [[sum(1 - adjacency_matrix[i][j] for j in range(n) if i != j) if i == j else -adja

    # Compute second-largest eigenvalue of the Laplacian matrix
    lambda_2 = second_largest_eigenvalue(laplacian)
    if lambda_2 is None:
        return {
            "metric_name": "resolution_length",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    # Construct Tseitin formula and run DPLL-based SAT solver
    resolution_length = run_dpll_solver(laplacian)
    if resolution_length is None:
        return {
            "metric_name": "resolution_length",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "solver_failed"
        }

    # Verify the conjecture
    c = Fraction(1, 2)  # Example constant, adjust as needed
    if math.log2(resolution_length) >= c * lambda_2 * n:
        return {
            "metric_name": "resolution_length",
            "metric_value": resolution_length,
```

```
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "resolution_length",
            "metric_value": resolution_length,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Counterexample: lambda_2={lambda_2}, n={n}, resolution_length={res
        }

def second_largest_eigenvalue(matrix):
    def power_iteration(matrix, v, max_iter=1000):
        for _ in range(max_iter):
            v = [sum(matrix[i][j] * v[j] for j in range(len(v))) for i in range(len(v))]
            norm = math.sqrt(sum(x**2 for x in v))
            if norm == 0:
                return None
            v = [x / norm for x in v]
        return v

    def rayleigh_quotient(matrix, v):
        numerator = sum(matrix[i][j] * v[i] * v[j] for i in range(len(v)) for j in range(len(v)))
        denominator = sum(v[i]**2 for i in range(len(v)))
        if denominator == 0:
            return None
        return numerator / denominator

    def find_second_largest_eigenvalue(matrix, max_iter=1000):
        v = [random.random() for _ in range(len(matrix))]
        lambda_1 = None
        lambda_2 = None

        for _ in range(max_iter):
            v = power_iteration(matrix, v)
            if lambda_1 is None:
                lambda_1 = rayleigh_quotient(matrix, v)
            else:
                lambda_2 = rayleigh_quotient(matrix, v)
                if lambda_2 is not None and (lambda_2 > lambda_1 or (lambda_2 == lambda_1 and lamb
                    lambda_1, lambda_2 = lambda_2, lambda_1

        return lambda_2

    return find_second_largest_eigenvalue(matrix)

def run_dpll_solver(laplacian):
    # Placeholder for DPLL solver implementation
    # This is a dummy function and should be replaced with actual DPLL code
    return None

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
```

```
results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results; insufficient data to evaluate support fraction or counterexamples. | next: Increase timeout duration and re-run experiment with stricter parameter monitoring

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	67237
2	preregistration	ollama_remote	qwen3:8b	0	0	27693
3	novelty	ollama_remote	qwen3:8b	0	0	24104
4	novelty	ollama_remote	qwen3:8b	0	0	20694
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15660
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13711
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15812

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14079
9	judge	ollama_remote	qwen3:8b	0	0	18126

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 217115 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/815cd97daf8d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/815cd97daf8d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/815cd97daf8d.tar.gz` (if generated)